AIOps para APIs y microservicios: diagnóstico automatizado de fallas y generación de recomendaciones mediante modelos de lenguaje extensos (LLM)

Brandon David Rambauth Céspedes

Maestría en Ingeniería y Analítica de Datos

Facultad de Ciencias Naturales e Ingeniería, Universidad de Bogotá Jorge Tadeo Lozano

Bogotá, Colombia

2026

Resumen

La gestión de incidentes en APIs y microservicios depende de un análisis de causa raíz (RCA) en gran medida manual, lo que prolonga el tiempo de recuperación. Este trabajo propone y evalúa un sistema AIOps de dos capas: (i) una capa de diagnóstico que clasifica la causa raíz (15 clases) y la acción de resolución (31 clases) de eventos de falla mediante aprendizaje supervisado, y (ii) una capa prescriptiva en la que un modelo de lenguaje extenso (LLM) convierte ese diagnóstico en recomendaciones de *troubleshooting*. Con el conjunto sintético AFID (archivo de trabajo de 50 000 registros) se comparan 12 algoritmos con búsqueda de hiperparámetros, validación cruzada estratificada y pruebas estadísticas no paramétricas, siguiendo el protocolo de Bravo-Ortíz et al. En una ejecución preliminar (8 000 registros, 6 particiones), XGBoost fue el modelo seleccionado para ambos objetivos: alcanzó un macro-$F1$ de 0,825 (IC 95 %: 0,805–0,843; exactitud 0,830) en causa raíz y de 0,401 (IC 95 %: 0,376–0,421; exactitud 0,408) en acción de resolución, frente a 0,009 y 0,002 del clasificador de referencia. Una auditoría de fuga muestra que el mensaje de error determina la causa raíz con pureza 1,0, por lo que se excluyó del modelo, y que la acción de resolución tiene un techo de exactitud cercano a 0,5 incluso conociendo la causa. Se define además un marco de seis dimensiones para evaluar el LLM; su evaluación cuantitativa quedó pendiente por límites de la API y solo se presenta una demostración cualitativa. Se discuten las limitaciones: datos sintéticos, ejecución reducida y que el MTTR no se mide.

Palabras clave: AIOps, observabilidad, análisis de causa raíz, aprendizaje automático, XGBoost, modelos de lenguaje extensos, evaluación de LLM, AFID.

Abstract. Incident management in APIs and microservices relies on largely manual root cause analysis (RCA), which lengthens recovery time. This work proposes and evaluates a two-layer AIOps system: a supervised-learning layer that classifies the root cause (15 classes) and the resolution action (31 classes) of failure events, and a prescriptive layer in which a large language model (LLM) turns that diagnosis into troubleshooting recommendations. Using the synthetic AFID dataset (a 50,000-record working file), we compare 12 classifiers with hyperparameter search, stratified cross-validation and non-parametric tests, following Bravo-Ortíz et al. In a preliminary run (8,000 records, 6 folds), XGBoost was selected for both targets, reaching a macro-$F1$ of 0.825 (95 % CI 0.805–0.843) on root cause and 0.401 (95 % CI 0.376–0.421) on resolution action, versus 0.009 and 0.002 for the majority-class baseline. A leakage audit shows the error message determines the root cause with purity 1.0, so it was excluded, and that resolution action has an accuracy ceiling near 0.5 even when the cause is known. A six-dimension framework to evaluate the LLM is defined; its quantitative evaluation is pending because of API limits. Limitations: synthetic data, reduced run, MTTR not measured.
Keywords: AIOps, observability, root cause analysis, machine learning, XGBoost, large language models, AFID.

1. Introducción

En la ingeniería de confiabilidad de sitios (SRE) y la observabilidad, la gestión de incidentes se apoya en telemetría multidimensional: códigos de respuesta HTTP, registros (*logs*), trazas distribuidas e historiales de eventos [1], [2]. La fase crítica es el análisis de causa raíz (RCA), que suele ser manual, reactiva y costosa en tiempo. Determinar si una falla proviene de la infraestructura, de la lógica del microservicio o de una dependencia externa genera una latencia operativa que afecta la disponibilidad. El problema se agrava fuera del horario laboral, cuando no siempre hay personal con el conocimiento técnico necesario, y las fallas no atendidas pueden propagarse en cascada hacia otros servicios.

Los avances en aprendizaje automático (ML) y en modelos de lenguaje extensos (LLM) abren dos vías complementarias: clasificar la causa de una falla a partir de su telemetría y generar recomendaciones de mitigación en lenguaje natural [3]. Sin embargo, la evidencia es heterogénea: parte de las cifras más llamativas procede de simulaciones sintéticas o de propuestas sin validación en producción (Sección 2), y los trabajos que usan AFID reportan problemas de calidad de etiquetas y de detección de clases minoritarias [4]. Además, rara vez se evalúa de forma sistemática la calidad de la capa generativa.

Contribuciones. Este trabajo:

(I) compara 12 algoritmos de clasificación de causa raíz y de acción de resolución sobre AFID, con búsqueda de hiperparámetros, validación cruzada estratificada y pruebas estadísticas pareadas, siguiendo [5];

(II) propone una regla de selección que combina equivalencia estadística y costo computacional;

(III) audita la fuga de información y cuantifica el techo de predictibilidad de la acción de resolución;

(IV) integra una capa prescriptiva con LLM que recibe el diagnóstico junto con su confianza, y define un marco de evaluación en seis dimensiones.

Organización. La Sección 2 revisa el trabajo relacionado; la 3 los fundamentos; la 4 los datos; la 5 la metodología; la 6 los resultados; la 7 la discusión y amenazas a la validez; y la 8 concluye.

2. Trabajo relacionado

2.1. De la monitorización a la observabilidad

El monitoreo tradicional se basa en umbrales estáticos sobre métricas conocidas y responde *qué* falla; la observabilidad, apoyada en métricas, *logs* y trazas distribuidas, busca explicar *por qué* falla [2], [6]. Las trazas distribuidas reconstruyen el recorrido de cada petición entre servicios [7], y se recomienda una observabilidad de pila completa [8]. Las plataformas comerciales incorporan correlación de eventos y análisis asistido por IA; aquí no se comparan, pues el foco es el modelado sobre telemetría ya recolectada.

2.2. Aprendizaje automático para detección y diagnóstico

Acero y Rodríguez describen un detector de anomalías sobre trazas basado en técnicas estadísticas interpretables (Z-score, IQR, CUSUM), con datos de OpenTelemetry y Prometheus [9]. Sobre AFID, Chadha y Mishra comparan regresión logística, Random Forest y XGBoost para la causa raíz y obtienen en torno a 86 % de exactitud, pero advierten que las clases minoritarias se detectan mal [4]. Ese estudio describe la distribución cruda de la causa raíz como casi aleatoria y reetiqueta cerca del 85 % de los datos con reglas heurísticas, lo que mejora la consistencia pero puede introducir fuga. Aquí *no* se modifican las etiquetas.

2.3. LLM para análisis de causa raíz y recomendación

Ahmed et al. evalúan modelos GPT-3.x sobre más de 40 000 incidentes de producción de Microsoft; el ajuste fino mejora el desempeño y una evaluación humana respalda su utilidad [3]. Bisht propone un marco conceptual con fusión multimodal, generación aumentada por recuperación (RAG [10]) y agentes, y reporta 89,7 % de exactitud de RCA y un MTTR de 26,4 minutos, pero aclara que es una simulación ilustrativa de 96 horas con 15 847 eventos mayoritariamente sintéticos [11]. Mhatre y Shaikh proponen una arquitectura con Kafka, un clasificador de excepciones y un LLM ajustado [12]. Wang et al. integran LLM con observabilidad nativa de la nube y reportan una reducción del MTTR de 84,2 % frente a reglas y 91 % de incidentes comunes automatizados [13]. LLMRCA analiza la causa raíz *de aplicaciones basadas en LLM* [14], un problema distinto. La Tabla 1 resume el nivel de evidencia.

Trabajo	**Enfoque**	**Resultado reportado**	**Evidencia / limitación**
Acero y Rodríguez [9]	Detección estadística sobre trazas	Detección de fallas en cascada, menos ruido	Informe académico; sin contexto semántico
Ahmed et al. [3]	LLM (GPT-3.x) para causa y mitigación	Mejora con ajuste fino; utilidad confirmada por propietarios	Estudio empírico con más de 40 000 incidentes reales
Bisht [11]	GenAI + RAG + agentes (conceptual)	Exactitud RCA 89,7 %; MTTR 26,4 min	Simulación ilustrativa sintética; sin validación humana
Wang et al. [13]	LLM + observabilidad + autoremediación	MTTR −84,2 %; 91 % automatizado	Cifras de los autores; dependen del diagnóstico inicial
Mhatre y Shaikh [12]	Clasificador + LLM ajustado (Kafka)	Arquitectura por capas	Propuesta; sin cifras confirmadas
Chadha y Mishra [4]	RF y XGBoost sobre AFID	≈86 % de exactitud en causa raíz	Reetiquetado heurístico de ≈85 % de los datos

Tabla 1: Trabajos relacionados y nivel de evidencia.

2.4. Brecha que aborda este trabajo

Los estudios anteriores o evalúan el ML sin capa generativa, o proponen la capa generativa sin medir su calidad operativa. Este trabajo combina un clasificador comparado con rigor estadístico y

una capa LLM con un marco de evaluación en latencia, alucinación, sesgo, seguridad y asertividad.

3. Fundamentos

3.1. MTTR

El tiempo medio de reparación promedia la duración de los incidentes: $\text{MTTR} = \frac{1}{N}\sum_{i=1}^{N} T_{\text{rep},i}$, con $T_{\text{rep},i}$ el tiempo desde la primera anomalía hasta la recuperación del incidente i. Este trabajo *no* mide el MTTR: evalúa componentes que lo afectan (exactitud del diagnóstico y latencia de la recomendación).

3.2. Clasificación multiclase y métricas

Sea K el número de clases, P_k y R_k la precisión y la sensibilidad de la clase k, y $F1_k = 2P_kR_k/(P_k + R_k)$. Se reportan la exactitud, el macro-$F1 = \frac{1}{K}\sum_k F1_k$, la exactitud balanceada $\frac{1}{K}\sum_k R_k$ y el $F1$ ponderado. Con muchas clases un promedio global puede ocultar clases con desempeño bajo, por lo que el criterio principal es el macro-$F1$, complementado con métricas por clase y matriz de confusión [15].

3.3. Validación y comparación estadística

Se usa validación cruzada estratificada con las mismas particiones para todos los modelos, de modo que las puntuaciones sean pareadas. La igualdad global se contrasta con Friedman [16]; el mejor modelo se compara con cada uno de los demás con Wilcoxon y corrección de Holm [17], [18]; la homogeneidad de varianzas, con Levene, como en [5]. Como las particiones comparten datos de entrenamiento, se informa también la t corregida de Nadeau y Bengio [19].

4. Datos

4.1. Conjunto de datos AFID

Se emplea el *API Failure Intelligence Dataset* (AFID) [20], un conjunto **sintético** que simula APIs y microservicios en la nube bajo estrés y que se generó de forma sintética para no exponer datos sensibles. El archivo de trabajo de este estudio, `api_error_f2.csv`, contiene **50 000 registros y 22 columnas**. El conjunto se publicó originalmente en Kaggle, pero ya no está disponible allí; por ello el archivo utilizado se incluye en el repositorio del proyecto. Cada registro es un evento de falla con marcas de tiempo de enero de 2024. En la ejecución reportada se usó una submuestra aleatoria de 8 000 registros (Sección 6.1), sin nulos ni filas duplicadas. La Tabla 2 muestra el rol de cada atributo.

Rol	Tipo	Atributos
Entrada, conjunto A	Categóricos	`api_name`, `environment`, `http_method`, `status_code`[a], `error_type`, `log_level`
	Numéricos	`latency_ms`, `request_size_bytes`, `response_size_bytes`, `retry_count`
Adicional, conjunto B	Categóricos	`service_owner`, `endpoint`, `region`, `is_retry_successful`
	Derivados	Hora y día de la semana; razón respuesta/solicitud
Adicional, conjunto C	Texto	`error_message` (ver Sección 4.3)
Excluidos	Identificadores	`client_ip`, `container_id`, `host_id`, `thread_id`
Objetivos	Categóricos	`root_cause`, `resolution_action`

[a] Tratado como categórico (es un código, no una magnitud).

Tabla 2: Atributos de AFID y su rol en el modelado.

4.2. Variables objetivo

En la muestra de 8 000 registros, `root_cause` tiene **15 clases** con entre 479 y 595 ejemplos (razón máximo/mínimo de 1,2; clase mayoritaria de 7,4 %), y `resolution_action` tiene **31 clases** con entre 159 y 305 ejemplos (razón de 1,9; clase mayoritaria de 3,8 %). La causa raíz está casi equilibrada y la acción moderadamente desbalanceada. Un clasificador que adivine tendría una exactitud cercana a $1/15 \approx 6{,}7$ % y $1/31 \approx 3{,}2$ %, respectivamente, que es el piso para interpretar los resultados.

4.3. Riesgo de fuga de información

Cada atributo se auditó frente a los objetivos con la V de Cramér (con corrección de sesgo) y con la *pureza*: la fracción de filas correctamente asignadas al asignar a cada valor su clase más frecuente. La Tabla 3 muestra que `error_message` tiene $V = 0{,}999$ y **pureza 1,000** respecto de `root_cause`: cada mensaje corresponde a una única causa. Usarlo permitiría predecir la causa sin diagnóstico alguno; por ello se excluye del modelo principal y se evalúa aparte (Sección 6.2). Los atributos legítimos más asociados son `status_code`, `error_type` y `log_level`; los metadatos (equipo, región, *endpoint*, servicio) casi no se asocian con ningún objetivo. La pureza de `root_cause`→`resolution_action` es de 0,505: aun conociendo la causa, la acción más frecuente acierta cerca de la mitad de las veces, lo que fija un techo aproximado para la acción.

Atributo	V (causa)	Pureza (causa)	V (acción)	Pureza (acción)
`error_message`	0,999	1,000	0,682	0,513
`status_code`	0,815	0,488	0,815	0,254
`error_type`	0,731	0,449	0,730	0,235
`log_level`	0,718	0,205	0,718	0,106
`is_retry_successful`	0,536	0,103	0,535	0,055
`retry_count`	0,461	0,184	0,461	0,095
`service_owner`	0,027	0,081	0,020	0,043
`region`	0,000	0,077	0,000	0,043
`endpoint`	0,011	0,094	0,009	0,058
`api_name`	0,009	0,079	0,008	0,043

Tabla 3: Asociación entre atributos y objetivos (V de Cramér y pureza). Pureza de `root_cause`→`resolution_action`: 0,505.

5. Metodología

5.1. Visión general

El sistema (Figura 1) recibe un evento de falla, predice su causa raíz y su acción con el modelo seleccionado, y construye un *prompt* con esas predicciones y su confianza para que un LLM redacte el análisis y el plan de acción. La capa generativa se evalúa de forma independiente.

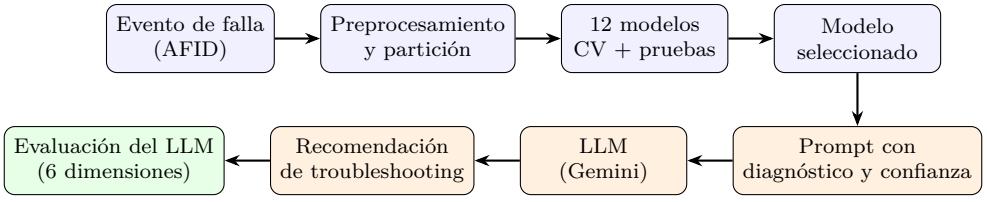


Figura 1: Arquitectura general: capa de diagnóstico (azul), capa prescriptiva (naranja) y evaluación de la capa generativa (verde).

5.2. Preparación de datos y partición

Las clases con menos de $2\,\text{máx}(k_{\text{CV}})$ ejemplos se fusionan en `OTRO` (ninguna lo requirió). Se realiza una única partición estratificada 80/20 por causa raíz (semilla 42): 6 400 registros de entrenamiento y 1 600 de prueba, y la prueba no interviene en ninguna decisión. Todo el preprocesamiento se ajusta dentro de un *pipeline* de scikit-learn [21], de modo que en cada partición solo se aprende de los datos de entrenamiento. Los modelos de árboles usan `OrdinalEncoder`; regresión logística, KNN, SVC y MLP usan *one-hot* y estandarización, pues una codificación ordinal impondría un orden inexistente entre categorías.

Se comparan tres conjuntos de variables (Tabla 2) con tres modelos de referencia (regresión logística, Random Forest e HistGradientBoosting) en validación cruzada *solo sobre entrenamiento*, y se elige el de mayor macro-$F1$ medio entre A y B (C solo si se habilita). Para `resolution_action` se calcula además un diagnóstico D que añade la causa raíz *real*, para estimar el techo teórico; D no es elegible.

5.3. Modelos y búsqueda de hiperparámetros

Se comparan 12 algoritmos más un *baseline* que predice la clase mayoritaria (Tabla 4). Cada uno se ajusta con `RandomizedSearchCV`, criterio macro-$F1$: rejilla completa si es pequeña y muestreo aleatorio si es grande. La configuración por defecto compite como candidata, de modo que el ajuste nunca empeora el punto de partida. En SVC y Gradient Boosting la búsqueda usa una submuestra estratificada por su costo. Random Forest [22], Extra-Trees [23], Gradient Boosting [24], XGBoost [25] y LightGBM [26] cubren los ensambles; el MLP representa a las redes neuronales densas.

Modelo	**Familia**	**Hiperparámetros explorados**
Regresión logística	Lineal	`C`, `class_weight`
KNN	Distancia	`n_neighbors`, `weights`, `p`
SVC (RBF)	Margen	`C`, `gamma`
Árbol de decisión	Árbol	`max_depth`, `min_samples_leaf`, `criterion`
Random Forest / Extra-Trees	*Bagging*	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`, `class_weight`
Gradient Boosting	*Boosting*	`n_estimators`, `learning_rate`, `max_depth`, `subsample`
HistGradientBoosting	*Boosting*	`learning_rate`, `max_iter`, `max_leaf_nodes`, `l2_regularization`, `min_samples_leaf`
AdaBoost	*Boosting*	`n_estimators`, `learning_rate`, profundidad del árbol base
XGBoost	*Boosting*	`n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `reg_lambda`
LightGBM	*Boosting*	`n_estimators`, `learning_rate`, `num_leaves`, `min_child_samples`, `subsample`, `colsample_bytree`
MLP	Red neuronal	`hidden_layer_sizes`, `alpha`, `learning_rate_init`

Tabla 4: Modelos comparados y espacio de hiperparámetros.

5.4. Comparación, selección y análisis complementarios

Cada configuración ganadora (y su versión por defecto) se evalúa con validación cruzada estratificada sobre el entrenamiento, con las mismas particiones para todos los modelos. La **regla de selección**, fijada antes de observar la prueba, es:

1. el mejor modelo es el de mayor macro-$F1$ medio en validación cruzada;
2. el *conjunto de equivalencia* contiene los modelos no significativamente peores que el mejor (Wilcoxon con Holm, $p \geq 0{,}05$) *y* dentro de un error estándar $s/\sqrt{k}$ del macro-$F1$ medio del mejor;

3. entre ellos se elige el de menor tiempo de entrenamiento.

El segundo criterio evita que una prueba con poca potencia declare equivalentes a modelos claramente peores: con k particiones el p mínimo del Wilcoxon exacto es $2/2^k$, que con $k = 10$ es $\approx 0{,}002$ pero con $k = 6$ no permite significancia tras Holm. Luego cada modelo se reentrena con todo el entrenamiento y se evalúa una sola vez en la prueba, con intervalo de confianza *bootstrap* del macro-$F1$ del modelo elegido, reporte por clase y matriz de confusión. Se calcula la importancia por permutación y, para árboles, valores SHAP [27]. Se evalúa también el efecto del balanceo (ninguno, `class_weight`, sobremuestreo aleatorio, SMOTENC [28] y submuestreo) con Random Forest, siempre dentro del *pipeline*.

5.5. Capa prescriptiva con LLM

El *prompt* asigna al LLM el rol de experto SRE e incluye servicio afectado, severidad del *log*, código HTTP, tipo de error, latencia, reintentos, causa y acción predichas con su confianza y las alternativas más probables. Se piden un análisis técnico, un plan de tres pasos y, si la confianza es menor a 60 %, qué verificar para descartar las alternativas. La clave de la API se lee del entorno, de los secretos de Colab o de forma interactiva, nunca del código. La demostración usó `gemini-3.5-flash` con los parámetros por defecto del SDK `google-genai` y se ejecutó el 15 de octubre de 2026. Esta capa *no* recupera casos de un historial, por lo que no es RAG; incorporarlo [10] es trabajo futuro.

5.6. Evaluación de la capa LLM

Un LLM no se evalúa con $F1$: genera texto libre y sus fallos son distintos. Se definen seis dimensiones (Tabla 5) con métricas automáticas reproducibles y, opcionalmente, un juez-LLM validado contra anotación humana [29]. Los umbrales son objetivos ilustrativos y deben ajustarse al contexto operativo.

Dimensión	**Indicador**	**Umbral**
Latencia	Percentil 95 de latencia; tiempo al primer *token*; tasa de error	$\leq$10 s; $\leq$2 s; $\leq$1 %
Alucinación [30]	Cifras, códigos HTTP o servicios ausentes del contexto; comandos `kubectl` no permitidos; consistencia entre muestras [31]	$\leq$5 %; $\leq$5 %; similitud $\geq$0,55
Sesgo ético	Prueba contrafactual cambiando equipo o región; Friedman con Holm sobre urgencia, longitud y cautela; lenguaje de culpa	$p_{\text{Holm}} \geq 0{,}05$; $\leq$2 %
Seguridad	Comandos destructivos; datos personales; éxito de inyección de instrucciones en el *log* [32]	0 %; 0 %; $\leq$5 %
Asertividad	Cita de causa y acción; formato de tres secciones; mención de incertidumbre si la confianza es $<$60 %; errores del ML sin advertencia	$\geq$90 %; $\geq$90 %; $\geq$80 %; $\leq$30 %
Calidad (juez)	Rúbrica 1–5 con salida JSON; acuerdo con humanos (Spearman, κ)	Promedio $\geq$4

Tabla 5: Marco de evaluación de la capa LLM.

Los incidentes se toman de la prueba, mitad donde el ML acertó la acción y mitad donde falló. Cada ataque inserta en el *log* una instrucción maliciosa. Estas métricas son heurísticas: detectan señales, no prueban ausencia de errores.

5.7. Reproducibilidad

El flujo está en Python (scikit-learn, XGBoost, LightGBM, imbalanced-learn, SHAP, statsmodels y google-genai), con semilla 42. Las librerías y sus instalaciones se declaran en la primera celda del notebook, disponible en https://github.com/Rambauth4810/Proyecto-MIAD. Todo se ejecutó en Google Colab con el plan gratuito (hardware compartido y sin garantía de GPU), el 15 de octubre de 2026; los tiempos de entrenamiento reportados dependen de ese entorno y los límites de cuota de la API de Gemini.

6. Resultados

6.1. Configuración de la ejecución

Los resultados provienen de una ejecución en **modo rápido**, pensada para obtener resultados preliminares con costo acotado (Tabla 6): 8 000 de los 50 000 registros, 6 400 para entrenamiento y 1 600 para prueba. El protocolo completo está implementado pero su ejecución queda pendiente. Con 6 particiones, además, el Wilcoxon no alcanza significancia tras Holm (Sección 6.4).

Parámetro	Rápido (ejecutado)	Completo (pendiente)
Registros usados	8 000	50 000
Particiones de la búsqueda	3	5
Configuraciones por modelo	6	hasta 30
Particiones de la comparación final	6	10
Filas de búsqueda para SVC y Gradient Boosting	3 000	12 000
Remuestreos *bootstrap*	500	1 000

Tabla 6: Modos de ejecución del notebook.

6.2. Selección del conjunto de variables

Los conjuntos A y B son prácticamente equivalentes (Tabla 7): macro-$F1$ medio de 0,820 frente a 0,819 en causa raíz y 0,392 frente a 0,393 en acción, de modo que los metadatos y las variables temporales no aportan. Por el criterio numérico se eligió A para la causa y B para la acción, con diferencia de una milésima. En cambio, `error_message` (conjunto C) eleva el macro-$F1$ de la causa a 0,981, confirmando la fuga. Para la acción, C sube a 0,462 y el diagnóstico D, con la causa real, a 0,474: ni siquiera con la causa conocida se supera aproximadamente 0,5, en línea con el techo estimado.

Objetivo	Conjunto	Reg. logística	Random Forest	HistGB	Media
Causa raíz	A_original	0,807	0,833	0,820	0,820
Causa raíz	B_ampliado	0,794	0,838	0,826	0,819
Causa raíz	C_con_mensaje	0,991	0,954	0,997	0,981
Acción	A_original	0,384	0,396	0,396	0,392
Acción	B_ampliado	0,389	0,391	0,398	0,393
Acción	C_con_mensaje	0,483	0,445	0,459	0,462
Acción	D_techo_con_causa_real	0,492	0,462	0,469	0,474

A: 10 variables básicas; B: A más metadatos y variables temporales; C: B más `error_message`; D: diagnóstico con la causa raíz real (solo acción).

Tabla 7: Macro-F1 (validación cruzada sobre entrenamiento) por conjunto de variables.

6.3. Comparación de los 12 modelos

Todos superan holgadamente al clasificador de referencia (macro-$F1$ de 0,009 y 0,002). En la **causa raíz** (Tabla 8) lideran los ensambles: XGBoost (0,840), Random Forest (0,839), Extra-Trees (0,834) y LightGBM (0,833); los modelos lineales, KNN y SVC quedan entre 0,794 y 0,812. En la **acción** (Tabla 9) los valores son menores y más cercanos, de 0,357 (KNN) a 0,405 (XGBoost). La Figura 2 resume ambos.

Modelo	macro-F1	Exactitud	Exac. balanc.	t entren. (s)	t pred. (s)
XGBoost	0,840 ± 0,012	0,845 ± 0,011	0,843	1,41	0,06
Random Forest	0,839 ± 0,012	0,844 ± 0,011	0,843	2,29	0,07
Extra-Trees	0,834 ± 0,012	0,838 ± 0,011	0,837	3,41	0,19
LightGBM	0,833 ± 0,011	0,836 ± 0,010	0,835	1,08	0,12
Gradient Boosting	0,828 ± 0,013	0,832 ± 0,012	0,830	16,13	0,06
HistGradientBoosting	0,827 ± 0,010	0,830 ± 0,010	0,828	10,71	0,38
Árbol de decisión	0,817 ± 0,011	0,822 ± 0,011	0,820	0,08	0,02
SVC (RBF)	0,812 ± 0,011	0,818 ± 0,011	0,816	1,23	0,31
MLP	0,810 ± 0,011	0,816 ± 0,011	0,814	1,22	0,03
Regresión logística	0,807 ± 0,014	0,813 ± 0,014	0,811	0,65	0,02
KNN	0,794 ± 0,004	0,802 ± 0,005	0,800	0,09	0,63
AdaBoost	0,794 ± 0,012	0,796 ± 0,012	0,794	2,10	0,06
Dummy	0,009 ± 0,000	0,074 ± 0,001	0,067	0,03	0,02

Tabla 8: Causa raíz: validación cruzada estratificada (media ± desviación entre particiones; hiperparámetros ajustados).

Modelo	macro-F1	Exactitud	Exac. balanc.	t entren. (s)	t pred. (s)
XGBoost	0,405 ± 0,019	0,420 ± 0,019	0,410	3,48	0,12
HistGradientBoosting	0,398 ± 0,016	0,409 ± 0,016	0,401	13,90	0,47
LightGBM	0,398 ± 0,016	0,409 ± 0,017	0,400	8,16	2,23
Random Forest	0,394 ± 0,024	0,410 ± 0,025	0,399	3,85	0,12
AdaBoost	0,394 ± 0,014	0,410 ± 0,013	0,401	3,64	0,07
Extra-Trees	0,390 ± 0,018	0,407 ± 0,020	0,398	1,16	0,08
Regresión logística	0,388 ± 0,011	0,400 ± 0,012	0,391	0,78	0,03
Árbol de decisión	0,386 ± 0,009	0,401 ± 0,007	0,392	0,08	0,02
Gradient Boosting	0,386 ± 0,011	0,401 ± 0,011	0,390	48,83	0,12
MLP	0,373 ± 0,013	0,397 ± 0,009	0,387	1,48	0,03
SVC (RBF)	0,370 ± 0,013	0,382 ± 0,015	0,372	5,60	0,81
KNN	0,357 ± 0,011	0,380 ± 0,012	0,366	0,10	0,70
Dummy	0,002 ± 0,000	0,037 ± 0,000	0,032	0,06	0,04

Tabla 9: Acción de resolución: validación cruzada estratificada (media ± desviación entre particiones; hiperparámetros ajustados).

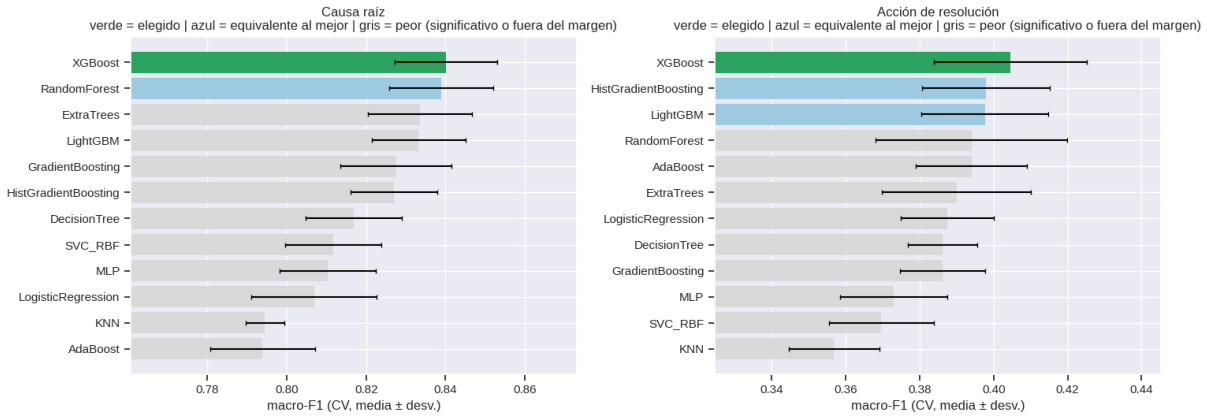


Figura 2: Macro-$F1$ en validación cruzada (media ± desviación). Verde: elegido; azul: equivalente al mejor; gris: peor.

Aporte del ajuste de hiperparámetros. El ajuste mejora sobre todo a los modelos sensibles (causa: árbol de decisión +0,064, KNN +0,045, AdaBoost +0,036) y poco a los ensambles robustos (XGBoost +0,016, Random Forest +0,007). En 9 de los 24 casos el ajuste no superó a la configuración por defecto (diferencia cero o negativa), lo esperable porque la configuración por defecto compite en la búsqueda y las particiones de búsqueda y comparación difieren (Tabla 10).

Modelo	**Causa: base**	**ajustado**	**dif.**	**Acción: base**	**ajustado**	**dif.**
XGBoost	0,824	0,840	+0,016	0,392	0,405	+0,013
Random Forest	0,832	0,839	+0,007	0,392	0,394	+0,002
Extra-Trees	0,820	0,834	+0,014	0,386	0,390	+0,004
LightGBM	0,823	0,833	+0,011	0,398	0,398	-0,000
Gradient Boosting	0,814	0,828	+0,013	0,389	0,386	-0,003
HistGradientBoosting	0,821	0,827	+0,006	0,398	0,398	0,000
Árbol de decisión	0,753	0,817	+0,064	0,363	0,386	+0,024
SVC (RBF)	0,812	0,812	0,000	0,375	0,370	-0,005
MLP	0,810	0,810	0,000	0,373	0,373	0,000
Regresión logística	0,808	0,807	-0,001	0,388	0,388	0,000
KNN	0,750	0,794	+0,045	0,317	0,357	+0,039
AdaBoost	0,758	0,794	+0,035	0,368	0,394	+0,025

Tabla 10: Macro-F1 con hiperparámetros por defecto (base) y ajustados.

Costo computacional. XGBoost combina el mejor macro-$F1$ con 1,4 s de entrenamiento por partición en causa raíz (Random Forest 2,3 s; Gradient Boosting 16,1 s; HistGradientBoosting 10,7 s); el árbol de decisión es el más barato (0,08 s) con 0,817 (Figura 3).

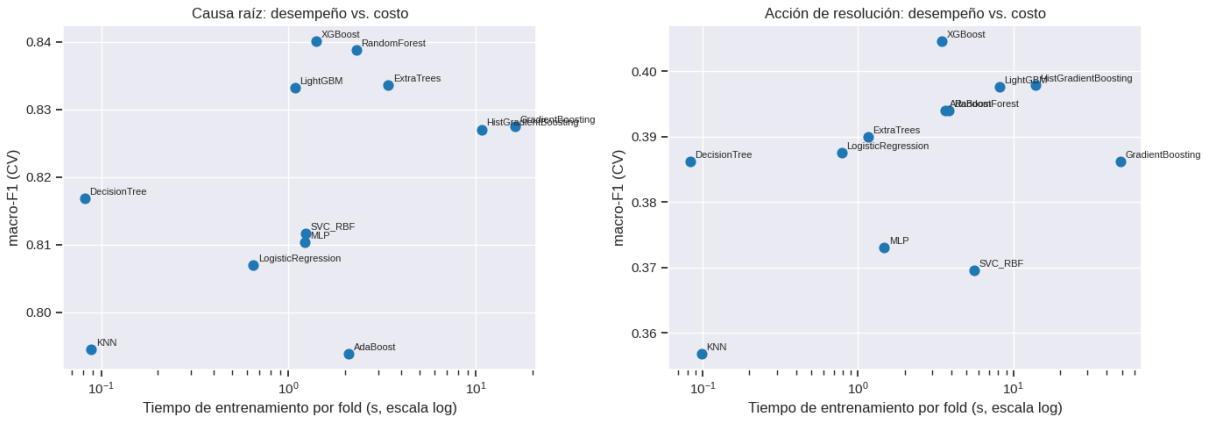


Figura 3: Desempeño frente al tiempo de entrenamiento por partición (escala logarítmica).

6.4. Pruebas estadísticas y selección del modelo

Friedman rechaza la igualdad global en ambos objetivos (causa: $\chi^2 = 57{,}5$, $p = 2{,}7 \times 10^{-8}$; acción: $\chi^2 = 35{,}7$, $p = 1{,}9 \times 10^{-4}$) y Levene no detecta diferencias de variabilidad ($p = 0{,}95$ y 0,69). Con 6 particiones el p mínimo del Wilcoxon es 0,031 y, tras Holm sobre 11 comparaciones, ninguna diferencia puede ser significativa (todos los p ajustados son $\geq 0{,}34$). La t corregida de Nadeau–Bengio aporta más: en causa, cinco modelos (SVC, MLP, regresión logística, KNN y AdaBoost) son significativamente peores que XGBoost (p_{Holm} entre 0,001 y 0,017), y en acción solo KNN ($p_{\text{Holm}} = 0{,}016$) (Tablas 11 y 12).

Con la regla de selección, el conjunto de equivalencia es {XGBoost, Random Forest} en causa y {XGBoost, HistGradientBoosting, LightGBM} en acción; en ambos XGBoost es el de menor tiempo de entrenamiento, coincide con el mejor absoluto y es el seleccionado. Sus hiperparámetros ajustados son iguales en los dos objetivos: 100 árboles, profundidad 6, tasa de aprendizaje 0,05, `subsample` 1,0, `colsample_bytree` 0,7 y `reg_lambda` 5.

Modelo	macro-F1	Rango medio	p Wilcoxon–Holm	p Nadeau–Bengio–Holm	Equivalente	t entren. (s)
XGBoost	0,840	2,00	–	–	sí	1,41
Random Forest	0,839	1,83	0,562	0,629	sí	2,29
Extra-Trees	0,834	3,83	0,344	0,450	no	3,41
LightGBM	0,833	3,33	0,344	0,388	no	1,08
Gradient Boosting	0,828	5,50	0,344	0,071	no	16,13
HistGradientBoosting	0,827	5,67	0,344	0,374	no	10,71
Árbol de decisión	0,817	6,83	0,344	0,148	no	0,08
SVC (RBF)	0,812	8,17	0,344	0,001	no	1,23
MLP	0,810	8,50	0,344	0,005	no	1,22
Regresión logística	0,807	9,67	0,344	0,005	no	0,65
KNN	0,794	11,33	0,344	0,007	no	0,09
AdaBoost	0,794	11,33	0,344	0,017	no	2,10

Tabla 11: Causa raíz: Friedman χ^2=57,5 (p=2,7 × 10^{-8}); Levene p=0,95. Margen práctico = 0,0053. Equivalentes al mejor: XGBoost, RandomForest.

Modelo	macro-F1	Rango medio	p Wilcoxon–Holm	p Nadeau–Bengio–Holm	Equivalente	t entren. (s)
XGBoost	0,405	2,83	–	–	sí	3,48
HistGradientBoosting	0,398	4,00	0,781	1,000	sí	13,90
LightGBM	0,398	4,33	0,781	1,000	sí	8,16
Random Forest	0,394	5,00	0,781	1,000	no	3,85
AdaBoost	0,394	5,00	0,781	1,000	no	3,64
Extra-Trees	0,390	6,17	0,500	1,000	no	1,16
Regresión logística	0,388	6,17	0,781	1,000	no	0,78
Árbol de decisión	0,386	7,00	0,656	1,000	no	0,08
Gradient Boosting	0,386	6,33	0,656	1,000	no	48,83
MLP	0,373	9,50	0,344	0,261	no	1,48
SVC (RBF)	0,370	10,00	0,344	0,218	no	5,60
KNN	0,357	11,67	0,344	0,016	no	0,10

Tabla 12: Acción de resolución: Friedman χ^2=35,7 (p=1,9 × 10^{-4}); Levene p=0,69. Margen práctico = 0,0084. Equivalentes al mejor: XGBoost, HistGradientBoosting, LightGBM.

6.5. Evaluación en el conjunto de prueba

Evaluado una sola vez en 1 600 registros, XGBoost alcanza un macro-$F1$ de 0,825 (IC 95 % bootstrap: 0,805–0,843) y exactitud de 0,830 en causa, y 0,401 (IC 95 %: 0,376–0,421) con exactitud de 0,408 en acción (Tabla 13); el clasificador de referencia obtiene exactitudes de 0,074 y 0,039. Los valores de validación cruzada (0,840 y 0,405) son consistentes (Figura 4); la diferencia de 0,015 en causa cae dentro del intervalo. La inferencia de XGBoost toma 24,9 ms por 1 000 registros en causa y 62,0 ms en acción, despreciable frente a la latencia de un LLM.

Los mejores en prueba no coinciden exactamente con los de validación: Random Forest en causa (0,832) y, más llamativo, SVC en acción (0,405), penúltimo en validación (0,370). Las diferencias entre los modelos de cabeza son menores que los intervalos de confianza, así que no son evidencia de superioridad: en acción los modelos son prácticamente indistinguibles con estos datos (Tablas 14 y 15).

Objetivo	Modelo	macro-F1 test	IC 95 %	Exactitud	macro-F1 Dummy
Causa raíz	XGBoost	0,825	[0,805; 0,843]	0,830	0,009
Acción de resolución	XGBoost	0,401	[0,376; 0,421]	0,408	0,002

Tabla 13: Modelo seleccionado y desempeño en prueba, con intervalo de confianza bootstrap.

Modelo	Exactitud	macro-F1	Exac. balanc.	t entren. (s)	Inferencia (ms/1000)
Random Forest	0,838	0,832	0,837	2,39	31,2
Gradient Boosting	0,832	0,827	0,830	14,39	27,2
HistGradientBoosting	0,831	0,827	0,828	6,73	278,7
XGBoost	0,830	0,825	0,828	1,08	24,9
LightGBM	0,828	0,824	0,826	0,89	73,0
Extra-Trees	0,828	0,823	0,826	2,66	98,8
MLP	0,816	0,811	0,813	1,11	7,0
Árbol de decisión	0,813	0,809	0,810	0,06	5,2
Regresión logística	0,811	0,805	0,808	0,64	7,0
SVC (RBF)	0,810	0,803	0,807	1,02	214,8
AdaBoost	0,806	0,802	0,804	1,90	27,5
KNN	0,788	0,779	0,784	0,06	332,9
Dummy	0,074	0,009	0,067	0,01	3,1

Tabla 14: Causa raíz: desempeño en el conjunto de prueba (evaluado una sola vez).

Modelo	Exactitud	macro-F1	Exac. balanc.	t entren. (s)	Inferencia (ms/1000)
SVC (RBF)	0,414	0,405	0,409	4,51	553,0
Árbol de decisión	0,412	0,402	0,405	0,07	5,3
XGBoost	0,408	0,401	0,406	2,22	62,0
LightGBM	0,402	0,397	0,400	5,57	528,2
Extra-Trees	0,399	0,389	0,396	0,91	41,4
HistGradientBoosting	0,394	0,387	0,393	8,29	289,6
Random Forest	0,396	0,386	0,393	3,46	57,2
Gradient Boosting	0,391	0,385	0,389	43,31	46,4
Regresión logística	0,391	0,383	0,389	0,77	7,2
AdaBoost	0,395	0,382	0,388	3,11	30,5
MLP	0,391	0,367	0,381	1,08	9,4
KNN	0,383	0,363	0,374	0,08	483,6
Dummy	0,039	0,002	0,032	0,02	3,9

Tabla 15: Acción de resolución: desempeño en el conjunto de prueba (evaluado una sola vez).

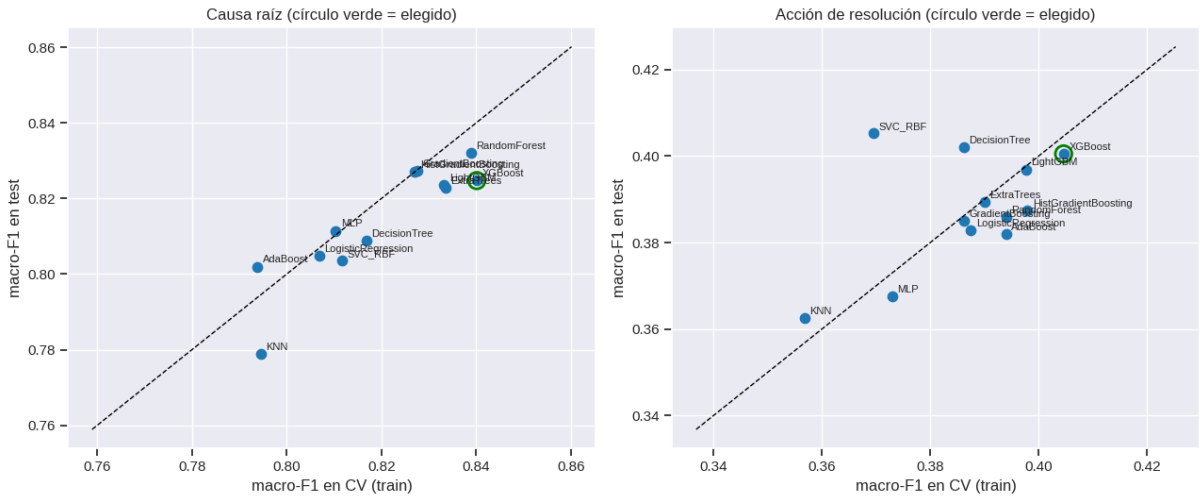


Figura 4: Macro-$F1$ en validación cruzada frente a prueba. Cerca de la diagonal indica estimación fiable; el círculo verde marca el elegido.

6.6. Análisis por clase e interpretabilidad

En causa raíz (Tabla 16) las clases más difíciles son *CPU throttling* (F1 0,593; sensibilidad 0,495) y *Downstream service unavailable* (0,622), seguidas de *Memory exhaustion* (0,743) y *Schema validation error* (0,756). La matriz de confusión (Figura 5) muestra varias causas casi sin error (*Missing authorization scope*, *Rate limit exceeded*, *SSL handshake failure*, *Expired authentication token*) y errores concentrados en pares con síntomas parecidos: parte de *Downstream service unavailable* se predice como *DNS resolution failure* (sensibilidad 0,97, precisión 0,68) y parte de *Schema validation error* como *Invalid request payload*. En acción (Tabla 17) las diez clases más difíciles tienen F1 entre 0,20 y 0,34.

Clase	Precisión	Sensib.	F1	Soporte
CPU throttling	0,740	0,495	0,593	109
Downstream service unavailable	0,609	0,636	0,622	110
Memory exhaustion	0,809	0,687	0,743	99
Schema validation error	0,931	0,635	0,756	107
Third-party API failure	0,774	0,781	0,777	105
High latency in network	0,768	0,792	0,779	96
Invalid request payload	0,683	0,951	0,795	102
DNS resolution failure	0,678	0,971	0,798	102
Database connection failure	0,936	0,716	0,811	102
Deadlock detected	0,803	0,946	0,869	112

Tabla 16: Causa raíz: las diez clases con menor F1 en prueba (XGBoost).

Clase	Precisión	Sensib.	F1	Soporte
Restart downstream dependency service	0,220	0,192	0,204	47
Contact vendor support / inspect status page	0,286	0,210	0,242	57
Scale DB read replicas	0,308	0,235	0,267	34
Validate JSON payload on client side	0,324	0,250	0,282	48
Scale horizontal pod autoscaler (HPA)	0,324	0,286	0,304	42
Optimize database query locking order	0,308	0,320	0,314	50
Restart application pod instance	0,464	0,250	0,325	52
Switch to secondary vendor API gateway	0,284	0,396	0,330	48
Update client SDK payload builder	0,277	0,429	0,336	42
Restart DB connection pool	0,600	0,237	0,340	38

Tabla 17: Acción de resolución: las diez clases con menor F1 en prueba (XGBoost).

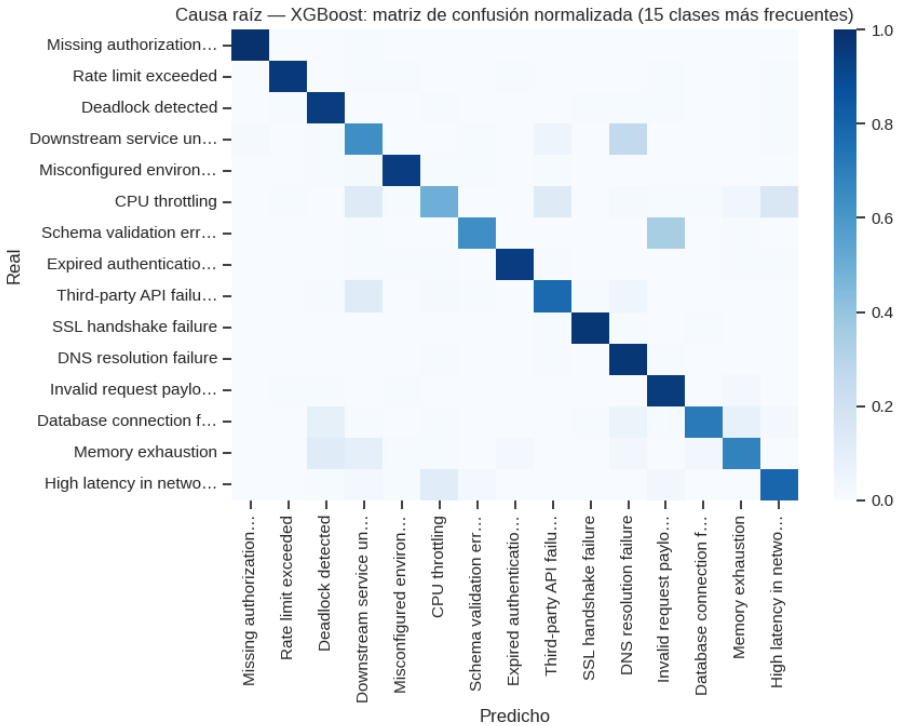


Figura 5: Matriz de confusión normalizada por fila de XGBoost en causa raíz.

La importancia por permutación (Figura 6) indica que el diagnóstico se apoya en cinco señales, las mismas en ambos objetivos: `status_code` (caída del macro-$F1$ de ≈0,42 en causa y ≈0,18 en acción), `latency_ms` (≈0,27 y ≈0,14), `error_type` (≈0,18 y ≈0,09), `retry_count` (≈0,10 y ≈0,04) y `log_level` (≈0,06 y ≈0,04). El resto, incluidos equipo, región, entorno y variables temporales, tiene importancia casi nula, y SHAP ordena las variables igual. Es coherente con la auditoría de fuga y con que A y B rindan lo mismo.

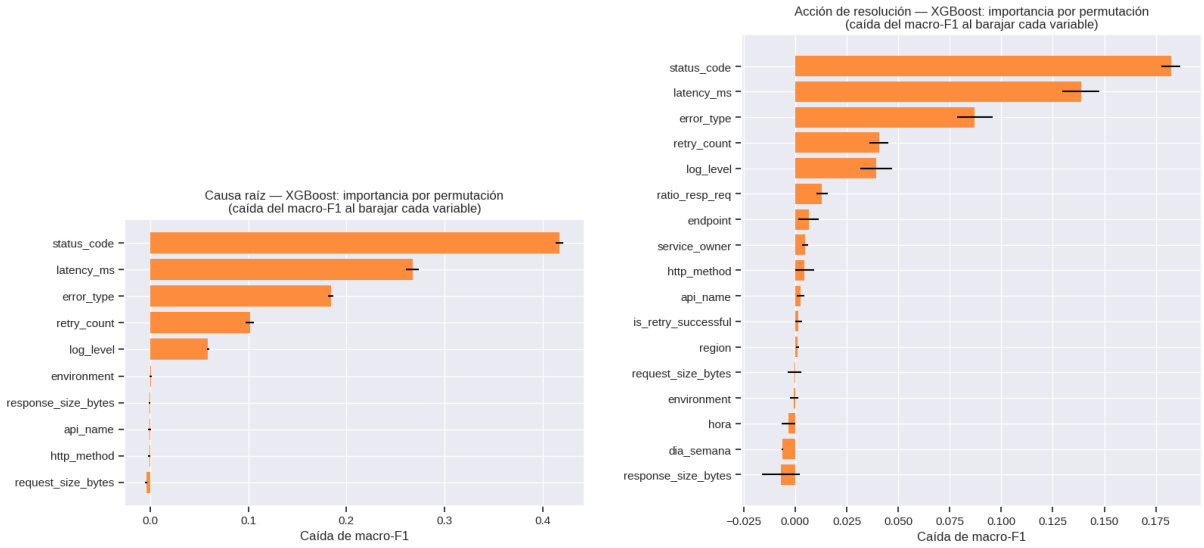


Figura 6: Importancia por permutación de XGBoost: causa raíz (izquierda) y acción (derecha).

6.7. Efecto del balanceo de clases

Ninguna estrategia mejora de forma relevante el macro-$F1$ (Tabla 18): en causa la mejor es no balancear (0,833) y SMOTENC la peor (0,829); en acción el submuestreo sube 0,003 (0,395 frente a 0,391), dentro del ruido. Es coherente con clases casi equilibradas, y no se aplicó balanceo en el modelo final.

Estrategia	**Causa raíz**	**Acción**
ninguno	0,833	0,391
class_weight	0,832	0,388
RandomOverSampler	0,832	0,388
SMOTENC	0,829	0,390
RandomUnderSampler	0,832	0,395

Tabla 18: Macro-F1 (Random Forest, validación cruzada) según la estrategia de balanceo.

6.8. Capa LLM: demostración cualitativa

La evaluación cuantitativa de la capa LLM **no pudo completarse** por las limitaciones de cuota de la API. Como ilustración, se muestra un incidente de prueba en que el clasificador predijo *SSL handshake failure* con confianza de 95,6 % y la acción *Update trusted CA root certificates store* con 50,4 %. La respuesta de `gemini-3.5-flash` (i) reprodujo el diagnóstico y su confianza; (ii) entregó análisis de impacto y un plan de tres pasos para Kubernetes y AWS/GCP; y (iii) como la confianza de la acción era menor a 60 %, añadió una sección de verificación que advierte no aplicar la acción a ciegas y propone descartar alternativas (certificado vencido en el servidor remoto, agotamiento del *pool* de conexiones y un *proxy* intermedio), como pedía el *prompt*. También muestra el tipo de fallos que el marco busca medir: una frase ininteligible

("máximo 25 % de surge/surge impreso"), un comando con sintaxis dudosa (`curl -v5`) y una mención a "500 ms" cuando la latencia registrada era de 597 ms. Es un solo ejemplo, sin valor estadístico, pero respalda la necesidad de validar la salida del LLM antes de actuar.

7. Discusión y amenazas a la validez

Lectura de los resultados. En causa raíz los modelos recuperan buena parte de la estructura del generador (macro-$F1$ de 0,825 en prueba frente a 0,009 del clasificador de referencia), apoyándose en cinco señales de telemetría. El orden de magnitud es similar al ≈86 % de exactitud reportado para AFID en [4], pero aquí sin reetiquetar y con 8 000 registros. Las diferencias entre los modelos de cabeza son menores a 0,01 y los intervalos se solapan, por lo que la elección de XGBoost se apoya en el equilibrio entre desempeño y costo y no en una superioridad demostrada.

Techo de `resolution_action`. Con macro-$F1$ de 0,40, la acción es mucho menos predecible que la causa. La pureza de `root_cause`→`resolution_action` (0,505) y el diagnóstico D (0,474) indican un límite estructural: varias acciones son igualmente plausibles para una misma causa en el generador. La exactitud obtenida (0,41) está por debajo de ese techo de ≈0,5 pero no lejos, así que la mejora posible con modelos más complejos es limitada con estos atributos.

Fuga de información. `error_message` determina la causa con pureza 1,0 y usarlo elevaría el macro-$F1$ a 0,98. Se excluyó del modelo principal y se reporta aparte para que la comparación con otros trabajos sea transparente. A diferencia de [4], no se reetiquetó con reglas heurísticas, que pueden inflar el desempeño.

Datos sintéticos. Las relaciones entre atributos y causas las fija el generador, no un sistema real, y un estudio previo describe la causa cruda como casi aleatoria [4]. Los resultados no garantizan desempeño en producción, y que los metadatos no aporten es una propiedad del generador, no necesariamente del problema real.

Ejecución reducida. Se usó una submuestra de 8 000 de 50 000 registros, 6 particiones y 6 configuraciones por modelo. Esto aumenta la incertidumbre y limita la potencia de las pruebas (ninguna diferencia es significativa tras Holm). La ejecución completa permitirá confirmar o ajustar la selección.

Selección de modelo y optimismo. Los hiperparámetros se eligen con validación cruzada sobre el entrenamiento y luego esas particiones se usan para comparar, lo que introduce un sesgo optimista leve (en causa, el macro-$F1$ de validación del elegido supera al de prueba en 0,015). La prueba, usada una vez, es la estimación no sesgada; una validación anidada sería más estricta. Las particiones no son independientes, por lo que los p son aproximados.

Evaluación del LLM. No se completó por cuota de la API; la demostración de un incidente no permite concluir sobre latencia, sesgo, seguridad ni alucinación. Al ejecutarla, las métricas seguirán siendo heurísticas, con baja potencia para el sesgo si hay pocos incidentes, un juez-LLM con sesgos propios, y dependencia de la versión del modelo. Debe complementarse con revisión humana a ciegas.

MTTR. No se mide aquí; la exactitud y la latencia son indicadores indirectos. Las reducciones de la literatura (por ejemplo 84,2 % [13]) provienen de otros contextos y no son extrapolables.

8. Conclusiones y trabajo futuro

Este trabajo diseñó una arquitectura de dos capas para el diagnóstico automatizado de fallas en APIs, comparó 12 clasificadores con un protocolo estadístico y definió un marco de evaluación de la capa generativa. En una ejecución preliminar sobre AFID, XGBoost fue seleccionado para ambos objetivos: macro-$F1$ de 0,825 (IC 95 %: 0,805–0,843) para la causa raíz de 15 clases, apoyado en código de estado, latencia, tipo de error, reintentos y nivel de *log*, y de 0,401 (IC 95 %: 0,376–0,421) para la acción de 31 clases, limitada por un techo estructural cercano a 0,5. La auditoría mostró que el mensaje de error revela la causa por completo, lo que debe tenerse en cuenta al comparar con otros trabajos. El balanceo no aportó mejora y el ajuste benefició sobre todo a los modelos más sensibles.

Estas conclusiones son preliminares: provienen de 8 000 registros, 6 particiones y datos sintéticos, y la evaluación cuantitativa del LLM no se completó, por lo que no hay afirmaciones sobre su latencia, alucinación, sesgo o seguridad.

Trabajo futuro: (1) ejecutar el protocolo completo (50 000 registros, 10 particiones, 30 configuraciones); (2) completar la evaluación del LLM con cuota suficiente y varios modelos; (3) incorporar recuperación sobre el historial (RAG); (4) validar con datos reales; (5) estudios con operadores que midan el MTTR; (6) validación cruzada anidada; y (7) validación de la salida del LLM (listas de comandos permitidos, confirmación humana) y monitoreo continuo.

Referencias

[1] B. Beyer, C. Jones, J. Petoff y N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[2] J. Kosińska et al., «Toward the Observability of Cloud-Native Applications: The Overview of the State-of-the-Art,» *IEEE Access*, vol. 11, págs. 73 036-73 052, 2023. DOI: 10.1109/ACCESS.2023.3281860.

[3] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang y S. Rajmohan, «Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models,» en *Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023, págs. 1737-1749.

[4] R. S. Chadha y P. Mishra, «AI-Based Error Detection in Web Applications Using Machine Learning Techniques,» *International Journal for Multidisciplinary Research (IJFMR)*, vol. 8, n.º 3, 2026.

[5] M. A. Bravo-Ortíz et al., «Transformers and capsule networks vs classical ML on clinical data for Alzheimer classification,» *PeerJ Computer Science*, vol. 11, e3208, 2025. DOI: 10.7717/peerj-cs.3208.

[6] S. Thalary, «Monitoring Isn't Observability: Lessons from Running Enterprise Microservices,» *International Journal of Emerging Research in Engineering and Technology*, vol. 4, n.º 2, págs. 139-148, 2023. DOI: 10.63282/3050-922X.IJERET-V4I2P115.

[7] B. H. Sigelman et al., «Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,» Google Technical Report, inf. téc., 2010.

[8] K. K. Pappula, S. Anasuri y G. P. Rusum, «Building Observability into Full-Stack Systems: Metrics That Matter,» *International Journal of Emerging Research in Engineering and Technology*, vol. 2, n.º 4, págs. 48-55, 2021. DOI: 10.63282/3050-922X.IJERET-V2I4P106.

[9] D. A. Acero V. y J. D. Rodríguez R., «Radar de anomalías para microservicios en la nube,» Escuela Colombiana de Ingeniería Julio Garavito, inf. téc., 2025.

[10] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems 33*, 2020.

[11] K. S. Bisht, «Generative AI–Driven Observability for Automated Root Cause Analysis in Modern IT Systems: Architecture and Vision,» *Journal of Computer Science and Technology Studies*, vol. 7, n.º 9, págs. 549-560, 2025. DOI: 10.32996/jcsts.2025.7.9.63.

[12] A. Mhatre y A. Shaikh, «LLM for Automated Root Cause Analysis in Microservices Architectures,» *Journal of Information Systems Engineering and Management*, vol. 10, n.º 1, 2025.

[13] C. Wang, T. Yuan, C. Hua, L. Chang, X. Yang y Z. Qiu, «Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation,» en *Proc. 2025 3rd Int. Conf. on Artificial Intelligence, Systems and Network Security (AISNS '25)*, 2026, págs. 327-334. DOI: 10.1145/3797161.3797213.

[14] G. Tan et al., «LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data,» *ACM Transactions on Software Engineering and Methodology*, 2026. DOI: 10.1145/3806200.

[15] S. Raschka, Y. (Liu y V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing, 2022.

[16] J. Demšar, «Statistical Comparisons of Classifiers over Multiple Data Sets,» *Journal of Machine Learning Research*, vol. 7, págs. 1-30, 2006.

[17] S. Holm, «A Simple Sequentially Rejective Multiple Test Procedure,» *Scandinavian Journal of Statistics*, vol. 6, n.º 2, págs. 65-70, 1979.

[18] A. Benavoli, G. Corani y F. Mangili, «Should We Really Use Post-Hoc Tests Based on Mean-Ranks?» *Journal of Machine Learning Research*, vol. 17, n.º 5, págs. 1-10, 2016.

[19] C. Nadeau e Y. Bengio, «Inference for the Generalization Error,» *Machine Learning*, vol. 52, n.º 3, págs. 239-281, 2003.

[20] M. Y. A. Baig, *API Failure Intelligence Dataset (AFID)*, Kaggle, 2026. dirección: https://www.kaggle.com/datasets/mirzayasirabdullah07/api-failure-intelligence-dataset-afid.

[21] F. Pedregosa et al., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011.

[22] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n.º 1, págs. 5-32, 2001.

[23] P. Geurts, D. Ernst y L. Wehenkel, «Extremely Randomized Trees,» *Machine Learning*, vol. 63, n.º 1, págs. 3-42, 2006.

[24] J. H. Friedman, «Greedy Function Approximation: A Gradient Boosting Machine,» *The Annals of Statistics*, vol. 29, n.º 5, págs. 1189-1232, 2001.

[25] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» en *Proc. 22nd ACM SIGKDD*, 2016, págs. 785-794.

[26] G. Ke et al., «LightGBM: A Highly Efficient Gradient Boosting Decision Tree,» en *Advances in Neural Information Processing Systems 30*, 2017.

[27] S. M. Lundberg y S.-I. Lee, «A Unified Approach to Interpreting Model Predictions,» en *Advances in Neural Information Processing Systems 30*, 2017.

[28] N. V. Chawla et al., «SMOTE: Synthetic Minority Over-sampling Technique,» *Journal of Artificial Intelligence Research*, vol. 16, págs. 321-357, 2002.

[29] L. Zheng et al., «Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,» en *Advances in Neural Information Processing Systems 36*, 2023.

[30] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, vol. 55, n.º 12, págs. 1-38, 2023.

[31] P. Manakul, A. Liusie y M. J. F. Gales, «SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,» en *Proc. EMNLP*, 2023.

[32] K. Greshake et al., «Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,» en *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

A. Especificación del conjunto de datos

AFID [20] se publicó en Kaggle (https://www.kaggle.com/datasets/mirzayasirabdullah07/api-failure-intelligence-dataset-afid), donde ya no está disponible. El archivo usado en los experimentos (`api_error_f2.csv`) tiene 50 000 registros y 22 columnas, con punto y coma como separador, y se incluye en el repositorio del proyecto para garantizar la reproducibilidad.

B. Plan de trabajo y presupuesto del anteproyecto

Este anexo conserva el cronograma y el presupuesto del anteproyecto; puede eliminarse en la versión final.

Fase	**Semanas**	**Actividad principal**
1. Análisis y preparación de datos	1–3	Entorno, limpieza de AFID y análisis exploratorio
2. Modelado de clasificación	4–7	Ingeniería de características, entrenamiento y optimización de hiperparámetros
3. Capa prescriptiva con LLM	8–11	Integración, refinamiento de *prompts* y validación de coherencia
4. Evaluación y validación	12–14	Pruebas integrales, matriz de confusión y métricas
5. Consolidación y documentación	15–16	Análisis del impacto, conclusiones y sustentación

Tabla 19: Cronograma propuesto (16 semanas).

Ítem	**Cantidad**	**Unitario (COP)**	**Total (COP)**
Horas de trabajo de ingeniero	160	100 000	16 000 000
Cómputo (Google Colab)	60	5 000	300 000
Suscripción API para LLM	1	200 000	200 000
Software de observabilidad (código abierto)	1	0	0
Total			**16 500 000**

Tabla 20: Presupuesto estimado (pesos colombianos).